The Stability Predictor:
What the Video Model Is, How It Was Pretrained, What We Train on Top of It, and the Training Plan on Our Labeled Data

Aryan Goyal

July 18, 2026 (updated July 20)

Abstract

This document describes the video model at the center of the project: the stability predictor. It explains what kind of model it is, how its backbone was pretrained, what the final trained layer looks like, exactly how its output is used next to the policy (and why it is never used inside policy training), the detailed plan for training it on our labeled stamps (about 155,000 labeled, now all in the training pool), and the measured results so far. Stage 1b: the pooled head reaches AUROC 0.87 on confident held-out stamps (0.89 with LIBERO added), single-task training does not transfer across tasks, and a single-frame DINOv2 ablation matches the video head almost exactly, so the signal is scene configuration rather than motion. A head trained on the closed-loop labels reaches only AUROC 0.627, so closed-loop expansion is at best weakly predictable from the same inputs. Stage 1c rollouts, three seeds pooled: the predictor-driven chunk switch ties the best fixed k and the best contact-oracle cell on tool_hang (0.800 against 0.807) while avoiding the per-step-replanning failure mode, and stays inside the flat band on the ceiling task square. The design principle throughout: the backbone stays frozen, the head is tiny, and the policy is never touched.

Contents

1 What the model is

The stability predictor is a video classifier with one job: look at the last 16 camera frames plus the robot's joint readings, and say whether the current moment is open-loop stable or open-loop unstable, with a number attached (the predicted divergence rate $\hat{\lambda}$).

It has two parts:

- **A frozen backbone:** V-JEPA 2, a ViT-L video transformer (checkpoint `facebook/vjepa2-vitl-fpc64-256` about 300 million parameters. We never update a single one of them. It converts a short video clip into a grid of feature tokens.
- **A trained head:** attentive pooling plus a two-layer MLP, a few million parameters. This is the only thing we train in the entire project. It reads the backbone's tokens and produces the stability outputs.

Why this shape: stability is a *dynamics* property. What makes a moment unstable is contact, incipient slip, and constrained motion, and these are *motion* cues. They are visible in a clip and mostly invisible in a single frame. So the backbone must understand motion, and V-JEPA 2 is pretrained specifically to predict motion (next section). A single-frame backbone (DINOv2) with the identical head is kept as the ablation that tests this claim: if the video model does not beat the single-frame model, the temporal story is wrong.

One clarification of naming, since it causes confusion: in this project the phrase "world model" is sometimes used loosely for this predictor. Strictly, the predictor does not model the world; it never generates or imagines future frames. It reads recent frames and classifies the present regime. A true action-conditioned world model (V-JEPA 2-AC or DINO-WM) enters the project only later, on real robots, as a replacement for the simulator in label generation (Section 9).

Why do we need a predictor at all? The ground-truth stability labels are indexed by demonstration and timestep, but at execution time the robot creates a brand new trajectory, visiting states that sit on no demonstration's timeline, so there is nothing to look up. The regime of the current moment must be judged live. In simulation this can be done exactly by freezing the state and running the perturb-and-replay probe; that oracle is used in Stage 1a to test whether the switching rule helps when given perfect information. A real robot cannot freeze time and rewind itself to measure its own divergence, and the only stability-relevant signal available at deployment is what the robot can see: the last second or two of camera frames. The predictor exists purely to read the regime out of those frames. If the oracle rule does not help, no predictor can; if it does, the predictor is what carries the rule out of the simulator.

2 How the backbone was pretrained

V-JEPA 2 is self-supervised on internet-scale video (on the order of a million hours of clips), with no labels and no pixel reconstruction. The objective is masked latent video prediction:

1. Take a video clip and hide large space-time blocks of it.
2. Encode the visible parts with the encoder.
3. A predictor network must predict the *representations* of the hidden blocks (as produced by a slowly-updated copy of the encoder), not their pixels.

Predicting representations instead of pixels matters: the model is not rewarded for memorizing textures or colors, it is rewarded for knowing *what will be where, and how things move*. To fill in a hidden block of frames correctly in latent space, the encoder has to carry information about object motion, contact, and continuity over time. That is precisely the information a stability readout needs. This is why we expect (and test) that a small head on top of these frozen features can read out "the gripper is about to bind against the peg" from 16 frames.

The specific checkpoint we use was pretrained at 256 by 256 resolution with 64-frame clips (the `fpc64-256` in the name); feeding it 16-frame clips at 256 resolution is a standard supported use. We use the encoder only and discard the pretraining predictor network.

3 The input and what the backbone produces

Per labeled stamp $(demo, t)$:

- **Video input:** the last 16 frames of the main camera ending at t (padded by repeating the first frame at episode start), resized to 256×256. Main camera: agentview for lift, can, square, and the MimicGen tasks; sideview for tool_hang; agentview for LIBERO.
- **Backbone output:** the encoder splits the clip into space-time patches (16 by 16 spatial patches, 2 frames per temporal slot, so $8 \times 16 \times 16$ tokens of dimension 1024). We average-pool each 4×4 spatial neighborhood, leaving $8 \times 4 \times 4 = 128$ tokens of dimension 1024 per stamp. This keeps coarse spatial layout and full temporal structure at 1/16 the storage.
- **Proprioception:** end-effector position (3), orientation quaternion (4), gripper joint positions (2), taken at t: a 9-dimensional vector.
- **Caching:** because the backbone is frozen, these features are computed once and stored (fp16). All four robomimic tasks are already extracted (the tool_hang set alone is 12 GB). Head training then never touches the encoder or the images, which is why it takes minutes per epoch instead of days.

An important practical note: the cached features are indexed by $(demo, t)$ and do not depend on the label values. When we regenerated all labels with the true measured σ_u, the features stayed valid; training joins features to the new labels by stamp index.

4 The final layer: the trained head

The head has three stages, all small:

1. **Attentive pooling.** A single learned query vector attends over the 128 tokens (standard cross-attention with a handful of heads) and produces one 1024-dimensional summary vector. In plain terms: the head learns *where and when to look* inside the clip. This is V-JEPA's own standard evaluation protocol (the "attentive probe"), reused unchanged.
2. **Proprio fusion.** The 9-dimensional proprio vector passes through a small MLP and is concatenated to the pooled summary.
3. **Output MLP.** A two-layer MLP maps the concatenation to four outputs:

- p_{ol}: probability the current moment is open-loop unstable (a sigmoid; this is the decision output),
- $\hat{\lambda}_{ol}$: predicted open-loop divergence rate (a regression; calibrates the head and enables a continuous-k rule as an ablation),
- p_{cl}, $\hat{\lambda}_{cl}$: the same pair for the closed-loop channel (trained from the start, used by the full three-case runtime rule and the mechanism analysis).

If the frozen-backbone head underfits, the planned fallback is LoRA adapters on the last encoder blocks. Nothing in the current results suggests this is needed.

5 How the head's output is used (and where it is NOT used)

This is the most commonly confused point, so it gets its own section.

The predictor's output is never used in policy training. The policy (Diffusion Policy now, ACT in stage 2) is trained entirely on its own, with its standard loss, finishes, and is then frozen forever. The predictor is also trained entirely on its own, supervised by the simulator labels. The two models never see each other during training.

They meet only at **execution time**. The policy always predicts a chunk of 16 future actions. The open question at every replan is how many of those predicted actions to execute before replanning. That number k is a queue-management knob outside the network. The predictor sets it:

Predicted regime	Executed chunk	Meaning
p_{ol} low (stable / not unstable)	$k = k_{\text{long}}$ (8–16)	commit; plant absorbs or carries errors, fewer injections
p_{ol} high, p_{cl} low	$k = 1$	react; feedback beats the expansion here
p_{ol} high, p_{cl} high	$k = 1$ + flag	no execution-side fix; segment marked for data-side repair

Hysteresis (two consecutive agreeing predictions) prevents chattering at segment boundaries. Because every compared execution rule runs the exact same frozen policy weights, any performance difference is attributable to the switching rule alone. That attribution is the experiment; baking the signal into policy training would destroy it.

The one place the predictor's output may eventually influence training is stage 2, and it is data-side, not loss-side: segments flagged by the third row (unstable under both channels) become the target list for noise-injected demonstrations, which is the theory's prescribed remedy for that regime.

6 Training plan on the datasets we have

6.1 The training pool

Supervised pairs are (cached features, labels) per stamp. Current inventory, all labeled with the perturb-and-replay oracle at K=24, N=8, stride 2:

Dataset	Stamps	Features	σ_u	Role in training
lift	2,485	cached	measured	pool
can	9,254	cached	measured	pool (mostly-stable balance)
square	12,723	cached	measured	pool + primary eval
tool_hang	45,633	cached	measured	pool + primary eval
MimicGen stack_d0	8,445	to extract	interim	pool (contact diversity)
MimicGen square_d0	12,874	to extract	borrowed	pool + transfer check
LIBERO-Long (10 files)	∼65k	to extract	interim	pool, provisional
policy rollouts (lift, can)	to label	to extract	measured	pool (runtime states)

Three preparation steps still pending, in order of priority: (1) label the collected lift and can policy rollouts (including the failed ones) exactly like demos, so the predictor sees the state distribution it will face at runtime, not only the demo tube; (2) regenerate image observations for the MimicGen sets (their downloads are low-dimensional) and extract features in the MimicGen environment stack; (3) extract LIBERO features (their files ship with images) with the extractor adapted to LIBERO's observation keys.

6.2 Targets and losses

- **Decision target:** the binary label unstable-versus-rest, taken AFTER deadband inheritance and median filtering. The measured regime proportions (about 30% unstable, near 0% confidently stable, 70% deadband) make a three-class target pointless; the runtime rule is binary anyway.
- **Decision loss:** class-weighted binary cross-entropy on p_{ol}. With a roughly 30/70 split the weighting is mild, but it is computed per dataset from the measured proportions rather than assumed.
- **Auxiliary regression:** Huber loss on $\hat{\lambda}_{ol}$ against the raw (unfiltered) λ, weight $\alpha = 0.5$. The raw value is used here because regression benefits from the continuous signal, including inside the deadband; the known tail noise in extreme λ values is why this is auxiliary and Huber rather than the decision loss and L2.
- **Closed-loop terms:** the same BCE + Huber pair on the closed-loop channel, weight $\beta = 0.5$, trained only on stamps where closed-loop labels exist (the stride-10 subsample on robomimic). Missing-label stamps simply contribute no CL gradient.

6.3 Splits and evaluation

- **Within-task split:** by demonstration, never by stamp (stamps 2 steps apart are nearly identical; splitting by stamp would leak). robomimic tasks use the official train/valid demo masks; other suites hold out 10% of demos.
- **Stage 1b metrics on held-out demos:** AUROC and calibration of p_{ol} against the oracle labels; agreement of $\hat{\lambda}$ with oracle λ (rank correlation); qualitative alignment with contact flags.
- **Cross-task transfer, the key generalization test:** train the head on Square only, evaluate label quality on Tool-Hang. This asks whether the model learned *stability* or a task-specific visual cue. The MimicGen variants add a second, easier transfer axis (human square to generated square).

- **Shortcut control:** shuffle-clip test (destroy temporal order, expect performance to drop toward the single-frame ablation) and the late-in-episode control (short tasks in the pool break the "late means contact" shortcut).
- **The temporal claim:** DINOv2 single-frame backbone with the identical head and data. The gap between it and V-JEPA 2 is the measured value of motion information.

6.4 Optimization details

AdamW on the head only (backbone frozen, features precomputed): learning rate 10^{-3} with cosine decay, weight decay 0.01, batch 512 stamps, up to 50 epochs with early stopping on validation AUROC. Balanced sampling across datasets so tool_hang's 45k stamps do not drown lift's 2.5k. On cached features one epoch over the full pool is minutes on one GPU; a full training run with ablations is an afternoon, not a compute event.

6.5 Order of experiments

1. Head on the four robomimic tasks (features ready today), Stage 1b metrics.
2. Add rollout-state stamps for lift and can; check whether runtime-state coverage moves held-out AUROC.
3. Cross-task transfer (Square to Tool-Hang) and the two ablations.
4. Add MimicGen, then LIBERO features to the pool; re-measure. LIBERO enters as provisional until its σ_u is measured with a LIBERO-trained policy.
5. Freeze the best head and hand it to Stage 1c: the predictor-driven k switch against fixed-k and policy-confidence baselines, all on the same frozen policy.

7 Stage 1b results (July 18)

The first two trainings from the experiment list have run to completion on cached features. Both trained the attentive head for 40 epochs, split by demonstration, with the class-weighted BCE on confident stamps plus the Huber regression on λ described above.

7.1 What was run

Four head trainings have completed. All share the same architecture (the attentive head of Section 5) and the same protocol: 40 epochs, batch 256, AdamW at learning rate 10^{-3} with cosine decay, split by demonstration with 20 percent of demos held out, seed 0.

Run 1, pooled Panda head. Eight feature sets: the four robomimic tasks (true measured σ_u labels), the two rollout-state sets (lift and can stamps collected from policy rollouts rather than demos), and the two MimicGen sets (stack_d0, square_d0). 82,212 train / 20,147 validation stamps, pooled $\delta = 0.0355$. This is the head used by Stage 1c.

Run 2, transfer test. Trained on square only (10,169 / 2,554, $\delta = 0.0372$), then evaluated frozen on the full tool_hang set (45,633 stamps) as the cross-task generalization test.

Run 3, Panda plus LIBERO, superseded. The run-1 pool plus all ten LIBERO files (131,282 / 34,243). Its train/validation split was drawn over the combined pool, which invalidated the comparison against run 1 (the leak note below); its numbers are kept only for the record.

Run 3b, Panda plus LIBERO, pinned validation. Same pool as run 3 (145,378 train stamps, pooled $\delta = 0.0326$) but validation pinned to run 1's exact 280 Panda demos (20,147 stamps), all LIBERO in training. This is the clean version of the LIBERO experiment.

7.2 As-run configuration, where it differs from the plan above

Three details of the plan in Section 7 were simplified in the code that actually ran. First, the BCE and Huber losses are summed with equal weight rather than the planned 0.5 factors; the BCE is class-weighted (positive weight from the confident training stamps) and computed on confident stamps only, and the Huber ($\delta_{huber} = 0.1$) covers all stamps including the deadband. Second, the closed-loop output channel is not trained yet: only about 1,300 closed-loop stamps exist so far (lift, can, square; tool_hang still labeling), too few to supervise a channel without overfitting, so the current head has two outputs (the unstable logit and $\hat{\lambda}_{ol}$) and the CL channel waits for more CL data. Third, there is no balanced dataset sampling and no early stopping; batches are drawn uniformly, so tool_hang and LIBERO dominate by stamp count, with the class-weighted BCE as the only correction. A balanced-sampling rerun is the natural follow-up if per-task validation ever shows the small tasks being neglected. Weight decay is 10^{-4}.

7.3 The numbers

Evaluation	AUROC (all)	AUROC (confident)	λ MAE	n
Run 1, held-out demos, pooled	0.699	**0.868**	0.027	20,147
Run 2, held-out demos, square	0.768	0.719	0.028	2,554
Run 2, transfer to tool_hang	0.562	0.465	0.072	45,633
Run 3, own mixed validation	0.646	0.880	0.025	34,243
Run 3b, own pinned validation	0.727	0.890	0.027	20,147
Run 1, pinned Panda val (same code path)	0.703	0.875	0.027	20,147
Run 3b, pinned Panda val (same code path)	**0.720**	**0.893**	0.027	20,147

The last two rows are the like-for-like LIBERO comparison: both heads scored by one evaluation script on the identical 20,147 Panda stamps that neither head trained on. Small differences against the training-time logs above them (0.868 versus 0.875 for run 1) come from the two code paths; comparisons are only ever made within one code path.

"All" scores every held-out stamp against $y = (\lambda > \delta)$; "confident" restricts to stamps outside the deadband ($|\lambda| > \delta$), which are the only stamps whose binary label is proven rather than ambiguous.

7.4 What the numbers mean

1. **The pooled head works where the labels are confident.** AUROC 0.868 on confident stamps means the head separates proven-unstable from proven-stable moments well from 16 frames plus proprioception. The lower all-stamps number (0.699) is expected by construction: the deadband stamps it scores against are ambiguous in the ground truth itself, so no predictor can score them cleanly.

2. **The regression channel is informative at the scale that matters.** λ MAE of 0.027 sits well below the labels' typical unstable magnitude (p95 around 0.11 to 0.13 across datasets), so thresholding $\hat{\lambda}$ against δ is meaningful and this is exactly how Stage 1c uses the head.

3. **Single-task training does not transfer.** Square to tool_hang lands at 0.562 overall and 0.465 on confident stamps, chance level or below, with the regression error tripling. A head trained on one task learns task-specific visual cues, not stability. The pooled head is therefore the only candidate for control, and the practical rule going forward is that every task family we care about contributes some labeled data to the pool; transfer is not relied on.

4. **Consequence for Stage 1c.** The frozen run-1 head, with its training $\delta = 0.0355$ as the default switching threshold, is the head handed to the Stage 1c predictor-driven chunk switch.

7.5 The LIBERO decision: in

The pooled retrain with all ten LIBERO files added (63k stamps, proprio harmonized to the 9-dimensional quaternion layout) was compared against run 1 on the identical 20,147 Panda validation stamps, with the validation demos pinned to run 1's exact split so the two heads saw the same training exposure. Result: AUROC 0.720 versus 0.703 on all stamps, 0.893 versus 0.875 on confident stamps, λ MAE unchanged. Adding LIBERO helps the Panda tasks modestly and hurts nothing, so LIBERO stays in the pool despite its provisional label quality.

One methodological note, recorded because it changed a number by a lot: the first version of this comparison drew the new head's train/validation split randomly over the combined pool, which put most of run 1's validation demos into the new head's training set. That leaked comparison showed 0.968, a large fake improvement. The pinned-split retrain shows the true gain, 0.018. The trainer now supports pinning the validation demos explicitly, and every future pool-variant comparison uses it.

7.6 The DINOv2 ablation: motion context adds almost nothing (July 20)

Run 4 asks how much of the head's signal actually comes from the 16-frame video clip. It replaces the V-JEPA 2 clip features with features from a single DINOv2-large frame at the query moment: one 84 to 240 pixel camera image, resized to 224, encoded to a CLS token plus an 8 by 8 pooled patch grid (65 tokens of dimension 1024). Stamps, proprioception, and labels are copied stamp-for-stamp from the run-1 pool, and the split is pinned to run 1's exact validation demos (82,212 train, 20,147 validation, same $\delta = 0.0355$), so the comparison is clean.

	AUROC all	AUROC confident	λ MAE
Run 1: V-JEPA 2, 16 frames	0.699	0.868	0.0268
Run 4: DINOv2, single frame	0.703	0.859	0.0269

The single frame matches the video clip almost exactly: the 15 extra frames of motion context are worth at most 0.009 confident AUROC. Three consequences. First, the stability signal the head reads is mostly scene configuration at the query moment (what is grasped, what is near what fixture) plus proprioception, not motion. Second, the shuffle-clip control is moot: if temporal order carried the signal, a single frame could not tie the clip, so that control is dropped. Third, for deployment a single-frame encoder would serve the switch nearly as well at a fraction of the inference cost.

One caveat on scope: the ablation changes encoder and clip length together (DINOv2 is a different pretraining), so it bounds the joint question "does the switch need a video encoder with motion context" (no, on these tasks) rather than isolating what V-JEPA 2 itself attends to.

7.7 Run 5: a head on the closed-loop labels (July 20)

With closed-loop labeling finished on all four robomimic tasks, run 5 trains the same head on the closed-loop expansion rate λ_{cl} instead of the open-loop label. V-JEPA 2 features were extracted at the 3,969 closed-loop stamps (151 lift, 477 can, 644 square, 2,697 tool_hang, so tool_hang is 68 percent of the pool), with the same architecture, losses, and 40 epochs on a 3,150 train, 819 validation split.

The label statistics change what every metric means. Closed-loop λ is positive at 97.6 percent of stamps, so the automatic threshold lands at $\delta = 0.076$, essentially the median of the labels (0.080). The classification target is therefore relative: does this stamp expand faster than the typical stamp, not stable versus unstable. The "confident" subset (true $|\lambda| > \delta$) contains only 7 negative stamps in the entire pool, so the reported confident AUROC of 0.986 is measured against a handful of contracting stamps and carries no weight.

The honest numbers are AUROC 0.627 over all validation stamps and a λ MAE of 0.043 against a threshold of 0.076. Ranking closed-loop expansion from one stamp's features is clearly harder than the open-loop task (0.699, though run 1 had 26 times the data, so part of the gap may be data volume). This is consistent with the mechanism: closed-loop expansion measures how the policy reacts to a perturbation, and that reaction depends on the policy, not only on the scene the encoder can see. Some signal transfers, since 0.627 is well above chance on 819 stamps, but nothing near a deployable switch. Nothing in the pipeline currently depends on this head; it exists to answer whether the closed-loop quantity is predictable from the same inputs, and the answer so far is only weakly.

8 Stage 1c rollout results (July 18 to 20)

Stage 1c puts the frozen run-1 head in the control loop: at every replan, the last 16 frames and the current proprioception go through the encoder and head, and the predicted $\hat{\lambda}$ picks the executed chunk length (k=16 if $\hat{\lambda} \leq \delta$, k=4 if $\hat{\lambda} > \delta$, with $\delta = 0.0355$ from training). Inference runs in a separate process with the newer transformers stack, about 100 ms per replan. Both tasks, 50 episodes, seed 0:

	Predictor (16,4)	Best oracle cell	Best fixed k	Fixed k=1
square	0.76	0.88	0.86	0.76
tool_hang	**0.82**	0.78	0.88	0.62

On square everything is inside the flat band and no conclusion follows, as expected for the ceiling task. Tool_hang is the informative row: the predictor cell beats every contact-oracle cell (0.82 against 0.56 to 0.78) and sits within one-seed noise of the best fixed k (0.88, gap 0.06 against a resolvable 0.16). Its successful episodes are also the fastest measured on the task (423 steps against 436 to 480 for all other cells).

The diagnostics say why. The predictor calls "unstable" on 71.6 percent of its decisions, well below the oracle's 84 to 88 percent contact rate, and its per-episode mean executed k is 11.4. It is not reproducing the binary contact signal; it is discriminating within contact, committing long

through contact stretches it judges benign. That is precisely the continuous-signal advantage the Stage 1a analysis said a binary oracle could not express, now visible in a rollout. What Stage 1c has not yet shown is the predictor beating the best fixed k; on these short contact-dominated tasks that may not be possible, and the long-horizon tasks are where that question gets a fair test. That test is now Stage 2 (four MimicGen tasks, policies training as of July 20); its design and status are in the Stage 2 companion document (`stage2_doc_latex`).

8.1 Seed repeats and the live probe (July 19 to 20)

The predictor cell was repeated at rollout seeds 1 and 2 on both tasks, and the live-probe oracle cell ran once at seed 0. Pooled over three seeds (n=150, one standard error about 0.033), with the pooled Stage 1a numbers as reference:

	Seeds (0,1,2)	Pooled	Best fixed	Best oracle	Fixed k=1
square	0.76 / 0.68 / 0.86	0.767	0.833	0.873	0.793
tool_hang	0.82 / 0.84 / 0.74	**0.800**	0.807	0.807	0.667

Tool_hang holds up under pooling: the predictor at 0.800 statistically ties both the best fixed k (0.807) and the best contact-oracle cell (0.807), while executing a mean k near 11 and avoiding the 0.60 to 0.67 trap of per-step replanning in contact. On square the predictor pools to 0.767, at the bottom of the flat band and about 1.4 standard errors below the best fixed cell; on the ceiling task the switch adds nothing, which is the expected null.

The live-probe cell (perturb-and-replay FTLE computed in the simulator at every replan, probe K=12, N=6, σ_u=0.05, threshold δ=0.05) came out 0.84 on square and 0.76 on tool_hang with mean executed k of 15.8 and 15.7. At that threshold the probe almost never fired, so the cell degenerates to fixed k=16 with probe overhead, and it is uninformative as an upper bound. Its probe protocol (K=12) also does not share a δ scale with the K=24 labeling protocol. Rerunning at a lower threshold would cost another day of GPU time for a bound the predictor cell already brackets in practice, so it is recorded and de-prioritized.

9 The later real-robot role of a true world model

On a real robot there is no simulator to reset and replay, so the label source changes: an action-conditioned world model (V-JEPA 2-AC, which adds action inputs to the same encoder family, or DINO-WM) imagines the paired rollouts in latent space. Same actions, perturbed first action, slope of the log latent divergence between the imagined futures: the same label, produced by imagination instead of simulation. The predictor's architecture, inputs, and losses do not change; only the oracle that writes the labels does. Hardware perturb-and-replay on a subsample is the fallback. This is deferred work and nothing in the current pipeline depends on it.

10 Summary

The model is a frozen 300M-parameter video transformer with a few-million-parameter trained head. The backbone learned motion by predicting masked video in latent space on internet-scale data; we never touch its weights. The head learns to read stability out of 16 frames plus proprioception, supervised by the labeled stamp pool (LIBERO included after the pinned-split comparison showed it helps). Stage 1b is now measured: the pooled head reaches AUROC 0.87 on

confident held-out stamps, single-task training does not transfer (0.56 across tasks), and a single-frame DINOv2 head matches the video head almost exactly, so the signal is scene configuration rather than motion. A head trained on the closed-loop labels reaches only 0.627, consistent with closed-loop expansion being a property of the policy's reaction rather than the visible scene. In rollouts (Stage 1c, three seeds pooled) the predictor switch ties the best fixed and best oracle cells on tool_hang at 0.800 while replanning adaptively, and stays inside the flat band on the ceiling task square. Its output never enters policy training; it gates, at execution time, how many actions of the frozen policy's predicted chunk are executed before replanning. Training the head is cheap by construction: every expensive thing (backbone pretraining, feature extraction, label generation) happens once, offline, and is already done.